

LOGLENS

SIEM-Lite Web Server Log Analysis and Threat Detection System

A Lightweight Security Analytics Platform for Automated Log Parsing and Attack Detection

Project Type: **Cybersecurity / Security Analytics**
Technology: **Web Security and Log Analysis**
Prepared By: **[Student Name]**
Department: **Computer Science and Engineering**
Institution: **[College Name]**
Academic Year: **2026**

Executive Summary

LogLens is a lightweight SIEM-lite system that automates web server log analysis for security threat detection. Web servers generate millions of log entries daily; manual analysis to identify SQL injection, XSS, directory traversal, and brute force attacks is impractical. LogLens provides an end-to-end pipeline: Upload Log -> Parse -> Detect Threats -> Aggregate -> Visualize -> Export Report. The system uses a streaming line-by-line parser with regex extraction, a signature database of 155+ attack patterns across 8 categories, composite threat scoring (0-100), MaxMind GeoIP2 geographic enrichment, and correlation analysis for multi-vector campaigns. Results are presented through a React dashboard with interactive visualizations and exportable PDF/CSV/JSON reports. Unlike enterprise SIEM platforms, LogLens is a focused, accessible tool requiring no external database.

1. Introduction

1.1 Background and Problem

Web servers produce logs recording every HTTP request: source IP, timestamp, method, path, status code, and response size. A server handling 1,000 requests/minute generates approximately 144,000 entries/day. Within this volume, attacks may represent less than 0.01% of traffic, yet missing them can mean a successful breach. Manual inspection, grep-based filtering, and static scripts are slow, require expertise, and fail to provide holistic visibility.

1.2 Objectives

- Parse Apache/Nginx CLF and Combined log formats using streaming regex extraction.
- Detect 8 attack categories: SQL injection, XSS, traversal, command injection, Log4Shell, recon, sensitive data, brute force.
- Aggregate events by source IP with composite threat scoring (0-100) and correlation analysis.
- Enrich with MaxMind GeoIP2 geographic data (Country, City, ASN).
- Present results via interactive React dashboard with PDF/CSV/JSON export.
- Process large files (500MB+) efficiently through streaming architecture.

2. System Architecture

LogLens follows a three-tier architecture: (1) Client Tier - React SPA with Tailwind CSS served by Vite; (2) Application Tier - Flask REST API managing uploads, background jobs, and report generation; (3) Processing Tier - streaming parser, detection engine, aggregation/scoring, and GeoIP enrichment. Self-contained: no external database required. Job results stored in-memory. Frontend build served as static files by Flask. Tech stack: Python 3.12, Flask 3.0.3, fpdf2, pandas, numpy, geoip2, React 18, TypeScript, Recharts, Tailwind CSS.

2.1 Data Flow

User uploads .log file -> Saved to temp directory -> Background thread spawned -> Line-by-line regex parsing -> Each record matched against 155+ signatures -> Events aggregated by source IP -> Threat scores calculated (severity 40%, frequency 25%, diversity 20%, recency 15%) -> GeoIP enrichment -> Correlation analysis -> Summary stored in job queue -> Dashboard polls for results -> PDF/CSV/JSON export.

3. Log Parser Engine

Parses Apache/Nginx Common Log Format using compiled regex. Example log line: 192.168.1.10 - - [20/Sep/2026:10:15:30 +0530] "GET /index.html HTTP/1.1" 200 1532. Extracted fields: IP, Timestamp, Method, Path, Protocol, Status, Size. Streaming architecture reads one line at a time (1KB memory per line) enabling constant memory usage. A 500MB file processes in approximately 3 minutes. Malformed lines are counted but do not halt processing.

4. Threat Detection Engine

Signature-based detection using structured JSON database with 155+ patterns across 8 categories. Each signature specifies type, severity, confidence, regex patterns, rule_id, and recommendation.

Attack Type	Detection Patterns	Severity
SQL Injection	UNION SELECT, OR 1=1, SLEEP(), information_schema	Critical
XSS	<script>, javascript:, onload, document.cookie	High
Dir. Traversal	../, ..%2f, /etc/passwd, .env, .git/config	High
Command Inject	; cat, ls, \$(cmd), backticks	Critical
Log4Shell	jndi:ldap, jndi:, CVE-2021-44228	Critical
Reconnaissance	sqlmap, nikto, nuclei, wp-login, xmlrpc	Medium
Sensitive Data	/.env, /backup, credentials, config files	Medium
Brute Force	5+ HTTP 401/403 from same IP in 60s window	High

Severity: Critical = active exploitation; High = confirmed attempts; Medium = suspicious activity; Low = informational. Each detection includes confidence, rule_id, and actionable recommendation.

5. Aggregation and Threat Scoring

Events consolidated by source IP into attacker profiles: total count, attack distribution, severity breakdown, geographic location, first/last seen. Composite threat scores (0-100) from weighted factors:

Factor	Weight	Description
Severity Distribution	40%	Proportion of critical/high severity events
Event Frequency	25%	Number of attacks relative to baseline
Category Diversity	20%	Number of distinct attack types used
Recency	15%	How recently the IP was active

Correlation analysis identifies multi-vector campaigns. GeoIP2 databases (Country, City, ASN) provide geographic enrichment. Brute force detected via behavioral rules (frequency + time window), not regex.

6. Frontend Dashboard

React 18 + TypeScript SPA with Tailwind CSS and Recharts. Dashboard sections: (1) Summary cards - total requests, attacks, malformed lines, attack rate; (2) Severity breakdown - color-coded chart; (3) Attack type distribution bar chart; (4) Geographic distribution table; (5) Top attacker profiles with IP, hits, country, city, attack, severity, ASN, last seen; (6) Per-attacker threat score bars; (7) Correlation findings cards; (8) Attack timeline - peak hours; (9) Recent events table; (10) Report export (PDF/CSV/JSON).

7. Existing vs Proposed System

Feature	Traditional Analysis	LogLens
Log Parsing	Manual / grep-based	Automated regex (streaming)
Threat Detection	Manual / scripts	155+ signature database
Large File Handling	Crash / slow / OOM	Streaming, constant memory
Visualization	Text / none	Interactive charts + tables
Attack Aggregation	Manual correlation	Automated IP profiling
Severity	Analyst judgment	Automated 4-level system
Threat Scoring	Not available	Composite 0-100 scores
LogLens Project Report v1.0 - Geographic	Not available	MaxMind GeoIP2 integration
Correlation	Not available	Multi-vector detection
Report Export	Manual	PDF, CSV, JSON automated

8. Requirements

8.1 Functional Requirements

Log upload (drag-and-drop), demo log loading, CLF/Combined parsing, 8-category attack detection, IP aggregation, threat scoring, correlation analysis, GeoIP enrichment, interactive dashboard, PDF/CSV/JSON export, job status tracking, malformed line handling.

8.2 Non-Functional Requirements

Performance: 100MB in under 60s, dashboard under 3s load. Scalability: up to 2GB via streaming. Reliability: graceful malformed line handling. Usability: no training required. Maintainability: modular component architecture. Security: input validation, rate limiting, CSP headers.

8.3 Software Requirements

Software	Version	Purpose
Python	3.10+	Backend runtime, log processing, threat detection
Flask	3.0.x	REST API, static file serving, CORS
fpdf2	2.8.x	PDF report generation
pandas / numpy	2.x / 1.24+	Data aggregation, numerical computation
geolip2	4.x	MaxMind GeoIP2 database queries
React	18.x	Frontend UI framework
TypeScript	5.x	Type-safe frontend development
Recharts	2.x	Data visualization library
Tailwind CSS	3.x	Utility-first CSS framework
GeoIP2 DBs	Latest	Country, City, ASN lookups

9. Testing and Validation

Comprehensive test suite with 51 unit tests covering all major components. Detection accuracy validated against curated test datasets with 100% pattern detection across all 8 categories. Integration tests verify the complete pipeline from upload to dashboard display.

Component	Tests	Coverage
Log Parser	8	CLF, Combined, malformed lines, edge cases
Detection Engine	12	All 8 attack categories, signature matching
Aggregator	10	IP grouping, severity calc, threat scoring
GeoIP Module	6	Country, City, ASN lookups, NaN handling
Threat Scoring	5	Score calculation, level classification
Correlation	4	Multi-vector, escalation, tool detection
API Endpoints	6	Upload, threats, reports, error handling

Edge cases tested: empty files, malformed lines, mixed valid/invalid data, very long lines, unicode characters, NaN/inf GeoIP values, duplicate IPs, no-attack scenarios. All 51 tests passing.

10. Performance Analysis

File Size	Log Lines	Processing Time	Peak Memory
10 KB	~100	under 1 second	~50 MB
1 MB	~10,000	~2 seconds	~50 MB
100 MB	~1,000,000	~45 seconds	~70 MB
500 MB	~5,000,000	~3 minutes	~90 MB
1 GB	~10,000,000	~6 minutes	~120 MB

Streaming architecture ensures constant memory usage regardless of file size. Throughput: 5,000-27,000 lines/second. Primary bottleneck: regex matching (CPU-bound for large files).

11. Security Considerations

- Input validation: All user inputs sanitized; file uploads validated for type and size (max 2GB).
- File security: SHA-256 hashes computed; temp directory cleaned by background thread.
- API security: Rate limiting, CORS restrictions, CSP/XSS/X-Frame headers.
- Data security: No external transmission; GeoIP lookups local; results cleared on restart.
- Known limitations: No authentication (prototype), no HTTPS by default, no persistent storage.

12. Future Enhancements

- Real-time log monitoring via file system watching (notify on Linux).
- Persistent storage with SQLite/PostgreSQL for historical analysis.
- Multi-file analysis with cross-file correlation.
- User authentication with role-based access control.
- Machine learning integration for anomaly detection beyond signatures.
- Webhook/email alerting for critical-severity events.
- Custom user-defined detection rules.
- Distributed processing with Celery/Redis for horizontal scaling.

13. Conclusion

LogLens demonstrates that a lightweight, focused security analytics tool can provide significant value for web server log analysis without enterprise SIEM complexity. The streaming parser efficiently handles files of any size, the 155+ signature database covers common attack vectors, and the aggregation engine adds analytical value through threat scoring and correlation. The React dashboard presents results intuitively, and export capabilities support documentation and programmatic consumption. LogLens fills an important gap: practical, automated log analysis for system administrators and small security teams. While it does not replace enterprise SIEM solutions, it delivers immediate value as a focused, accessible security analytics platform.

References

- [1] Apache HTTP Server Log Files. httpd.apache.org/docs/current/logs/
- [2] Nginx Logging Module. nginx.org/en/docs/http/nginx_http_log_module.html
- [3] OWASP SQL Injection. owasp.org/www-community/attacks/SQL_Injection
- [4] OWASP Cross-Site Scripting. owasp.org/www-community/attacks/xss/
- [5] MaxMind GeoIP2 Documentation. maxmind.com/en/geolite2/signup
- [6] Python 3 Documentation. docs.python.org/3/
- [7] Flask Web Framework. flask.palletsprojects.com/
- [8] fpdf2 PDF Generator. py-pdf.github.io/fpdf2/
- [9] React Documentation. react.dev/
- [10] CVE-2021-44228 (Log4Shell). nvd.nist.gov/vuln/detail/CVE-2021-44228
- [11] CWE-89: SQL Injection. cwe.mitre.org/data/definitions/89.html
- [12] CWE-79: Cross-Site Scripting. cwe.mitre.org/data/definitions/79.html
- [13] CWE-22: Path Traversal. cwe.mitre.org/data/definitions/22.html